

The Complete Graph Roadmap (Beginner → Advanced, Java)

88 curated problems, **every one unique across platforms** — LeetCode is used wherever a clean equivalent exists; GeeksforGeeks (8 problems) is used only for patterns LeetCode doesn't isolate cleanly as standalone drills (DFS-based cycle detection as distinct from LC's DSU-based versions, Prim's Algorithm explicitly, M-Coloring, Hamiltonian Path, Articulation Points, Kosaraju's SCC, Bellman-Ford negative-cycle detection). GfG problems show "—" in Problem # since only LeetCode assigns numeric IDs.

Tags: 🔥 Must-Do · ★ Recommended · 💡 Optional · 🏆 Hard Challenge · 📁 Premium

Level 0 — Graph Fundamentals & Representation

Pattern: Graph Fundamentals & Representation

Before problems: **build an adjacency list and an adjacency matrix by hand** for the same small graph, and know when each representation is better (adjacency list: sparse graphs, iterating neighbors; adjacency matrix: dense graphs, $O(1)$ edge-existence checks).

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
1	LeetCode	1971	Find if Path Exists in Graph	Easy	Fundamentals (BFS/DFS/Union-Find, pick any)	Build-your-own graph	🔥
2	LeetCode	997	Find the Town Judge	Easy	Fundamentals (in-degree/out-degree)	#1	★
3	LeetCode	1791	Find Center of Star Graph	Easy	Fundamentals (structural property)	#1	💡

Level 1 — BFS / DFS Basics

Patterns: BFS, DFS

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
4	LeetCode	200	Number of Islands	Medium	BFS/DFS 🔥 CORE	Level 0	🔥
5	LeetCode	733	Flood Fill	Easy	BFS/DFS	Level 0	★
6	LeetCode	133	Clone Graph	Medium	BFS/DFS + HashMap	#4	🔥
7	LeetCode	695	Max Area of Island	Medium	DFS	#4	🔥
8	LeetCode	130	Surrounded Regions	Medium	DFS from the boundary inward	#4	★
9	LeetCode	841	Keys and Rooms	Medium	DFS/BFS reachability	#6	🔥
10	LeetCode	1466	Reorder Routes to Make All Paths Lead to the City Zero	Medium	DFS on a tree treated as an undirected graph	#6	🔥

Must-Do & key pattern: #4 is the archetype — every problem in this level (and most of Level 2) reduces to "visit every reachable cell/node, mark it visited, recurse/enqueue into neighbors." #10 is worth dwelling on: it's given as directed edges but you must traverse it as if undirected (to reach every node) while remembering the *original* direction (to count "wrong-way" edges) — a good early lesson in "the traversal structure and the edge semantics can differ."

Level 2 — Grid & Matrix Graphs

Patterns: Number of Islands / Grid Graphs, Matrix BFS/DFS, Multi-Source BFS

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
11	LeetCode	1254	Number of Closed Islands	Medium	Grid DFS	Level 1	★
12	LeetCode	463	Island Perimeter	Easy	Grid Traversal	Level 1	💡
13	LeetCode	1905	Count Sub Islands	Medium	Grid DFS	#11	★
14	LeetCode	1020	Number of Enclaves	Medium	Grid DFS (boundary flood, then count)	#11	★
15	LeetCode	1091	Shortest Path in Binary Matrix	Medium	Matrix BFS (shortest path)	Level 1	🔥
16	LeetCode	1926	Nearest Exit from Entrance in Maze	Medium	Matrix BFS	#15	🔥
17	LeetCode	934	Shortest Bridge	Medium	DFS (mark island) + BFS (expand outward)	#15	🔥
18	LeetCode	994	Rotting Oranges	Medium	Multi-Source BFS 🔥 ARCHETYPE	#15	🔥
19	LeetCode	542	01 Matrix	Medium	Multi-Source BFS	#18	🔥
20	LeetCode	1162	As Far from Land as Possible	Medium	Multi-Source BFS	#18	★
21	LeetCode	417	Pacific Atlantic Water Flow	Medium	Multi-Source BFS (from two boundary sets)	#18	★
22	LeetCode	1765	Map of Highest Peak	Medium	Multi-Source BFS	#18	💡

Must-Do & key pattern: #18 (Rotting Oranges) is the second archetype of the roadmap — **seed the BFS queue with every "source" cell at once**, not one at a time; the level number the traversal finishes on is the answer directly. Every other multi-source problem here (#19–#22) is the same idea with a different seed set (all zeros, all land, all boundary cells, etc.).

Level 3 — Connected Components & Cycle Detection

Patterns: Connected Components, Cycle Detection — Undirected, Cycle Detection — Directed

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
23	LeetCode	547	Number of Provinces	Medium	Connected Components 🔥 CORE	Level 1	🔥
24	LeetCode	1319	Number of Operations to Make Network Connected	Medium	Connected Components	#23	🔥
25 🔒	LeetCode	323	Number of Connected Components in an Undirected Graph	Medium	Connected Components	#23	★
26 🔒	LeetCode	261	Graph Valid Tree	Medium	Connected Components + Cycle Detection combined	#23	★
27	LeetCode	684	Redundant Connection	Medium	Cycle Detection — Undirected (via DSU) 🔥 CORE	Level 0	🔥
28	GfG	—	Detect cycle in an undirected graph	Medium	Cycle Detection — Undirected (via DFS, contrast to #27's DSU)	#27	🔥
29	LeetCode	685	Redundant Connection II	Hard	Cycle Detection — Directed (advanced DSU)	#27	🏆
30	LeetCode	207	Course Schedule	Medium	Cycle Detection — Directed 🔥 CORE	Level 1	🔥
31	GfG	—	Detect Cycle in a Directed Graph	Medium	Cycle Detection — Directed (DFS recursion-stack / coloring technique)	#30	🔥
32	LeetCode	802	Find Eventual Safe States	Medium	Cycle Detection — Directed (reverse-graph view)	#30	★

Must-Do & key pattern: #27's Union-Find approach to cycle detection ("if two endpoints are already in the same set, this edge creates a cycle") and #28/#31's DFS approach ("a back-edge to an ancestor/recursion-stack node means a cycle") are **two genuinely different tools for the same job** — know both, because undirected cycle detection is usually cleanest with DSU while directed cycle detection needs the DFS-coloring technique (DSU doesn't directly generalize to directed edges).

Level 4 — Topological Sort & Kahn's Algorithm

Patterns: Topological Sort, Kahn's Algorithm

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
—	LeetCode	207	(cross-ref, Level 3)	Medium	Kahn's Algorithm (cycle detection via topo sort)	—	—
33	LeetCode	210	Course Schedule II	Medium	Kahn's Algorithm 🔥CORE (produce the order)	Level 3	🔥
34	LeetCode	310	Minimum Height Trees	Medium	Kahn's-style layer trimming (peel leaves inward)	#33	🔥
35	LeetCode	2115	Find All Possible Recipes from Given Supplies	Medium	Kahn's Algorithm (dependency resolution)	#33	🔥
36	LeetCode	1462	Course Schedule IV	Medium	Topological Sort + Reachability	#33	★
37	LeetCode	2192	All Ancestors of a Node in a DAG	Medium	Topological Sort + BFS/DFS	#33	★
—	LeetCode	802	(cross-ref, Level 3)	Medium	Topological Sort (reverse-graph Kahn's)	—	—
38 🔒	LeetCode	269	Alien Dictionary	Hard	Topological Sort (build the graph from constraints first)	#33	🔥
39	LeetCode	1857	Largest Color Value in a Directed Graph	Hard	Kahn's Algorithm + DP	#35	🏆
40	LeetCode	2050	Parallel Courses III	Hard	Topological Sort + DP (longest path in a DAG)	#33	🏆

Must-Do & key pattern: #33's **Kahn's Algorithm** is the queue-based way to topologically sort: compute in-degree per node, seed the queue with in-degree-0 nodes, decrement neighbors' in-degrees as you process each node, enqueue any that hit zero. Processing fewer nodes than the graph has means a cycle exists — this is the exact same idea as #30, just framed as "produce an order" instead of "detect impossibility." #38 (Alien Dictionary) is the hardest *construction* step in this level: you must derive the graph's edges yourself from adjacent word comparisons before Kahn's even starts.

Level 5 — Bipartite Graphs

Pattern: Bipartite Graph

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
41	LeetCode	785	Is Graph Bipartite?	Medium	Bipartite (2-coloring via BFS/DFS) 🔥 CORE	Level 1	🔥
42	LeetCode	886	Possible Bipartition	Medium	Bipartite (build the graph from "dislike" pairs first)	#41	🔥
43	LeetCode	1042	Flower Planting With No Adjacent	Medium	Graph Coloring (generalizes bipartite 2-coloring to 4 colors)	#41	★
44	LeetCode	1129	Shortest Path with Alternating Colors	Medium	Bipartite-flavored multi-state BFS	#41	★

Note: LeetCode's bipartite-graph coverage is genuinely this compact — these 4 problems isolate the pattern well without redundancy, so no padding was added here.

Must-Do & key pattern: #41 is 2-coloring: BFS/DFS from any uncolored node, alternating colors between adjacent nodes; if a conflict ever appears (two adjacent nodes forced to the same color), the graph isn't bipartite. #42 is the same algorithm wearing a different costume — you must build the adjacency list from pairwise constraints before the 2-coloring BFS/DFS even starts.

Level 6 — Shortest Path (Unweighted, Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS)

Patterns: Shortest Path — Unweighted, Dijkstra's Algorithm, Bellman-Ford, Floyd-Warshall, 0-1 BFS

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
—	LeetCode	1091	(cross-ref, Level 2)	Medium	Shortest Path — Unweighted	—	—
45	LeetCode	127	Word Ladder	Hard	Shortest Path — Unweighted (state-space BFS) 🔥 CORE	Level 2	🔥
—	LeetCode	1926	(cross-ref, Level 2)	Medium	Shortest Path — Unweighted	—	—
46	LeetCode	743	Network Delay Time	Medium	Dijkstra's Algorithm 🔥 CORE	Level 1, Heap basics	🔥
47	LeetCode	1631	Path With Minimum Effort	Medium	Dijkstra (modified relaxation: minimize the max edge, not the sum)	#46	🔥
48	LeetCode	1514	Path with Maximum Probability	Medium	Dijkstra (maximize a product instead of minimizing a sum)	#46	★
49	LeetCode	1976	Number of Ways to Arrive at Destination	Medium	Dijkstra + DP (count shortest paths, not just find one)	#46	🔥
50	LeetCode	778	Swim in Rising Water	Hard	Dijkstra-style minimax path	#47	★
51	LeetCode	2045	Second Minimum Time to Reach Destination	Hard	Dijkstra variant (second-shortest, not shortest)	#46	🏆
52	LeetCode	1928	Minimum Cost to Reach Destination in Time	Hard	Dijkstra + extra state (cost AND time constraint)	#46	🏆
53	LeetCode	787	Cheapest Flights Within K Stops	Medium	Bellman-Ford 🔥 CORE (bounded relaxations = bounded stops)	Level 1	🔥

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
54	GfG	—	Negative weight cycle	Medium	Bellman-Ford (cycle detection — the pattern Dijkstra <i>can't</i> do)	#53	🔥
55	LeetCode	1334	Find the City With the Smallest Number of Neighbors at a Threshold Distance	Medium	Floyd-Warshall 🔥CORE	Level 1	🔥
56	LeetCode	399	Evaluate Division	Medium	Floyd-Warshall / weighted DFS (all-pairs via given ratios)	#55	★
57	LeetCode	1368	Minimum Cost to Make at Least One Valid Path in a Grid	Hard	0-1 BFS 🔥 CORE	Level 2, #46	🔥
58	LeetCode	2290	Minimum Obstacle Removal to Reach Corner	Hard	0-1 BFS	#57	🔥

Must-Do & key pattern: #46 is the shortest-path archetype: a min-heap of `(distance, node)`, always expanding the currently-closest unvisited node, relaxing neighbor distances — works only with **non-negative** weights. #53's Bellman-Ford relaxes *all* edges up to k times, which is strictly weaker/slower than Dijkstra in general but handles negative weights and — as #54 shows — can detect a negative cycle (impossible to represent correctly with any "shortest distance so far," which is exactly the signal a negative cycle exists). #55's Floyd-Warshall computes **all-pairs** shortest paths in $O(V^3)$ using the beautifully simple recurrence `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])` for every intermediate `k` — reach for it specifically when you need distances between *every* pair, not just from one source. #57's 0-1 BFS reuses a plain `Deque` (push 0-weight destinations to the front, 1-weight to the back) to get Dijkstra-quality results in $O(V+E)$ whenever edge weights are only 0 or 1.

Level 7 — Union Find / DSU & Minimum Spanning Tree

Patterns: Union Find / DSU, Minimum Spanning Tree, Kruskal's Algorithm, Prim's Algorithm

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
—	LeetCode	684	(cross-ref, Level 3)	Medium	DSU	—	—
59	LeetCode	990	Satisfiability of Equality Equations	Medium	DSU 🔥 CORE	Level 0	🔥
60	LeetCode	721	Accounts Merge	Medium	DSU	#59	🔥
61	LeetCode	1202	Smallest String With Swaps	Medium	DSU	#59	★
62	LeetCode	1101	The Earliest Moment When Everyone Become Friends	Medium	DSU (time-ordered unions)	#59	★
63	LeetCode	947	Most Stones Removed with Same Row or Column	Medium	DSU (component-size counting)	#59	🔥
64	LeetCode	1061	Lexicographically Smallest Equivalent String	Medium	DSU	#59	★
65	LeetCode	2316	Count Unreachable Pairs of Nodes in an Undirected Graph	Medium	DSU (component-size math)	#63	★
66	LeetCode	1584	Min Cost to Connect All Points	Medium	Kruskal's / Prim's Algorithm 🔥 CORE	Level 7 DSU, Level 6 heap	🔥
67 🔒	LeetCode	1135	Connecting Cities With Minimum Cost	Medium	Kruskal's Algorithm	#66	★
68	LeetCode	1489	Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree	Hard	Kruskal's Algorithm (advanced: run MST with edges forced-in/forced-out)	#67	🏆
69	GfG	—	Minimum Spanning Tree	Medium	Prim's Algorithm (explicit heap-based practice)	#66	🔥

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
70 	LeetCode	1168	Optimize Water Distribution in a Village	Hard	Prim's/Kruskal's (virtual-node trick)	#69	

Must-Do & key pattern: #59 is the DSU archetype — `find` (with path compression) and `union` (by rank/size) give near- $O(1)$ amortized operations for "are these two things connected" and "connect these two things," which is why DSU quietly powers roughly a third of this entire roadmap. #66 is the bridge into MST: **Kruskal's** sorts all edges by weight and greedily unions the endpoints of each edge unless they're already connected (using DSU to check); **Prim's** instead grows a single tree outward, always adding the cheapest edge that connects the tree to a new node (using a min-heap) — know both, since interviewers sometimes specifically ask for one or the other. #70's "virtual node" trick (treating "the cost to dig a well" as an edge to an imaginary node 0) is a genuinely clever reduction worth remembering — it turns a "some nodes have a standalone cost" problem into a pure MST problem.

Level 8 — Advanced Graph Algorithms

Patterns: Backtracking on Graphs, Graph Coloring, Hamiltonian Path/Cycle, Bridges & Articulation Points, Strongly Connected Components

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
71	LeetCode	797	All Paths From Source to Target	Medium	Backtracking on Graphs	Level 1	🔥
72	GfG	—	M-Coloring Problem	Medium	Graph Coloring 🔥 CORE	Level 5, backtracking	🔥
73	GfG	—	Hamiltonian Path	Medium	Hamiltonian Path/Cycle 🔥 CORE	#71	🔥
74	LeetCode	980	Unique Paths III	Hard	Hamiltonian-style grid backtracking	#73	★
75	LeetCode	1192	Critical Connections in a Network	Hard	Bridges (Tarjan's Algorithm) 🔥 CORE	Level 1 DFS	🔥
76	GfG	—	Articulation Points (or Cut Vertices) in a Graph	Medium	Articulation Points 🔥 CORE	#75	🔥
77	GfG	—	Strongly Connected Components (Kosaraju's Algorithm)	Medium	SCC 🔥 CORE	Level 1 DFS	🔥
78	LeetCode	1568	Minimum Number of Days to Disconnect Island	Hard	Bridges/Articulation applied to a grid	#75, #76	🏆
79	LeetCode	2360	Longest Cycle in a Graph	Hard	Functional-graph cycle detection (SCC-adjacent)	#77	🏆

Must-Do & key pattern: #75's **Tarjan's Algorithm** tracks two numbers per node during a single DFS — its discovery time and the lowest discovery time reachable from its subtree ("low-link value") — and an edge (u, v) is a bridge exactly when $low[v] > disc[u]$ (v's subtree can't reach back to u or anything earlier). #76 (articulation points) uses the *same* DFS/low-link machinery with a slightly different condition. #77's Kosaraju's Algorithm finds SCCs with two DFS passes: one to compute a finishing-order stack on the original graph, one on the *transposed* (reversed-edges) graph processed in that stack's order. All three of these (#75, #76, #77) are variations of "one clever DFS pass with extra bookkeeping" — worth studying together rather than as three unrelated algorithms.

Level 9 — Graph + DP / Greedy / Heap

Patterns: Graph + Heap/Priority Queue, Graph + Greedy, Graph + Dynamic Programming

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
—	LeetCode	1631	(cross-ref, Level 6)	Medium	Graph + Greedy	—	—
—	LeetCode	1584	(cross-ref, Level 7)	Medium	Graph + Greedy (MST is inherently greedy)	—	—
80	LeetCode	2492	Minimum Score of a Path Between Two Cities	Medium	DSU + Greedy	Level 7	🔥
81	LeetCode	1519	Number of Nodes in the Sub-Tree With the Same Label	Medium	DFS + DP counting	Level 1	★
82	LeetCode	2246	Longest Path With Different Adjacent Characters	Hard	Tree DFS + DP	#81	🔥
83	LeetCode	847	Shortest Path Visiting All Nodes	Hard	BFS + Bitmask DP 🔥CORE	Level 6, bitmask basics	🔥
84	LeetCode	943	Find the Shortest Superstring	Hard	Hamiltonian Path + Bitmask DP	#83, Level 8	🏆
85	LeetCode	2493	Divide Nodes Into the Maximum Number of Groups	Hard	BFS + Bipartite Layering + Greedy	Level 5	🏆

Must-Do & key pattern: #83 is the pattern to master here — when you need the shortest walk visiting *every* node (not just one destination), plain BFS state (just "current node") isn't enough; the state becomes `(current node, bitmask of nodes visited so far)`, and BFS over that expanded state space still finds the shortest such walk. #82's tree DP tracks, for each node, the longest valid path found so far in its subtree while updating a global answer — the same "global variable + return-to-parent value" shape from the Trees roadmap's Diameter problem, just on a graph-shaped tree with a character-matching constraint layered on.

Level 10 — Advanced / Hard Interview Problems

Pattern: Advanced/Hard Graph Problems

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
—	LeetCode	269	(cross-ref, Level 4)	Hard	Advanced Topological Sort	—	—
—	LeetCode	1192	(cross-ref, Level 8)	Hard	Advanced Bridges	—	—
—	LeetCode	1489	(cross-ref, Level 7)	Hard	Advanced Kruskal's	—	—
—	LeetCode	2115	(cross-ref, Level 4)	Medium	Advanced Kahn's	—	—
—	LeetCode	1928	(cross-ref, Level 6)	Hard	Advanced Dijkstra	—	—
86	LeetCode	815	Bus Routes	Hard	BFS on a derived "graph of routes"	Level 2, Level 6	🏆
87	LeetCode	1595	Minimum Cost to Connect Two Groups of Points	Hard	Bitmask DP + Graph	Level 9	🏆
88	LeetCode	1263	Minimum Moves to Move a Box to Their Target Location	Hard	State-Space BFS (player + box position as one state)	Level 6	🏆

Must-Do & key pattern: #86 is a great final "aha" — naively BFS-ing stop-by-stop on the raw route network is too slow, so you instead build a graph *of routes* (two routes are "connected" if they share a physical stop) and BFS on that transformed graph instead. This whole level exists to reinforce one meta-skill: **recognizing when the graph as given isn't the graph you should actually search** — sometimes the real graph is over states (#88), route-pairs (#86), or bitmask-augmented nodes (#87, and #83/#84 from Level 9).

Graph Pattern Cheat Sheet

BFS vs DFS

- BFS explores level-by-level — use whenever you need the **shortest path / fewest steps** in an unweighted (or equal-weight) graph, because it discovers closer nodes strictly before farther ones.
- DFS explores as deep as possible before backtracking — better for exhaustive search, connectivity checks, cycle structure, and anywhere "shortest" doesn't matter, with lower constant-factor overhead than maintaining a queue.

- Rule of thumb: "shortest," "minimum steps," "fewest operations," "nearest" → BFS first.

Grid as a Graph

- Recognize: a 2D matrix where movement is allowed to 4 (or 8) neighboring cells.
- Core idea: treat each cell as an implicit graph node with up to 4/8 edges to its neighbors; use a `visit` `ed[][]` array or mutate the grid in place instead of an explicit adjacency list.

Connected Components

- Recognize: "how many groups," "are all nodes reachable from each other," "provinces/clusters."
- Core idea: run BFS/DFS (or DSU) from every unvisited node; each fresh traversal you have to start is one more component.

Cycle Detection

- Undirected: **DSU** — if an edge's two endpoints are already in the same set, adding it creates a cycle. Or DFS tracking a parent — revisiting a non-parent neighbor means a cycle.
- Directed: DSU does **not** directly work (union-find is inherently symmetric). Use **DFS with a recursion-stack / 3-color marker** (white/gray/black) — encountering a gray (currently-in-progress) node means a back edge, i.e., a cycle. Or use **Kahn's Algorithm**: if fewer nodes than the total get processed, a cycle exists.

Bipartite Graph

- Recognize: "can you split into two groups with no edge inside a group," "2-color the graph," "no two adjacent nodes share a property."
- Core idea: BFS/DFS, alternating a color between each node and its neighbors; a conflict (adjacent nodes forced to the same color) means the graph is not bipartite.

Topological Sort

- Recognize: "ordering with dependencies/prerequisites," on a **Directed Acyclic Graph (DAG)**.
- Two approaches: DFS-based (postorder, then reverse) or **Kahn's Algorithm** (queue-based, in-degree driven) — Kahn's is usually preferred in interviews since it naturally detects cycles as a side effect.

Shortest Path — algorithm choice

Situation	Algorithm	Why
Unweighted (or all edges equal weight)	Plain BFS	Level number = distance
Non-negative weights, single source	Dijkstra	Greedy + min-heap, $O((V+E) \log V)$
Negative weights allowed, single source	Bellman-Ford	Relax all edges $V-1$ times; $O(VE)$, also detects negative cycles
Need distances between every pair	Floyd-Warshall	$O(V^3)$ DP over intermediate nodes
Only two possible edge weights (0 and 1)	0-1 BFS	Deque, push 0-weight to front, 1-weight to back — $O(V+E)$

Dijkstra vs Bellman-Ford vs Floyd-Warshall

- Dijkstra: fastest, but **breaks with negative edge weights** (its greedy "never revisit a finalized node" assumption relies on non-negativity).
- Bellman-Ford: slower ($O(VE)$), but correct with negative weights, and its "did anything still improve after $V-1$ rounds" check is exactly how you detect a negative cycle.
- Floyd-Warshall: computes all-pairs at once — pick it when the graph is small enough (V^3 is fine) and you need many/all pairs of distances rather than one source.

0-1 BFS

- Recognize: shortest path where every edge costs 0 or 1 (e.g., "free to move this way, costs 1 to move that way").
- Core idea: a plain `Deque` instead of a priority queue — 0-weight destinations go to the front (processed before anything already queued), 1-weight destinations go to the back — preserves BFS's increasing-distance-order property in $O(V+E)$.

DSU (Disjoint Set Union / Union-Find)

- Recognize: "are these connected," "merge groups," "how many components after these unions," "detect a cycle while adding edges one at a time."
- Core idea: `find(x)` with path compression, `union(x, y)` by rank/size — both operations become near- $O(1)$ amortized (technically $O(\alpha(n))$, the inverse Ackermann function).

Kruskal vs Prim

- Both find a Minimum Spanning Tree, greedily, but from different angles:
- **Kruskal's**: sort *all* edges by weight; add each one unless it would create a cycle (checked via DSU). Best when edges are sparse relative to V^2 (easy to sort).
- **Prim's**: grow one tree from an arbitrary start node; repeatedly add the cheapest edge connecting the tree to a new node (via a min-heap). Best when the graph is dense (adjacency-matrix-friendly) or you want to build the tree incrementally from a specific root.

SCC (Strongly Connected Components)

- Recognize: **directed** graph, "mutually reachable," "condense into a DAG of components."
- Core idea (Kosaraju's): DFS once to get a finishing-time order; DFS again on the **transposed** graph, processing nodes in reverse-finish order — each DFS tree in the second pass is exactly one SCC. (Tarjan's Algorithm finds the same answer in a single DFS pass using low-link values, at the cost of being trickier to implement correctly.)

Bridges / Articulation Points

- Recognize: "critical connections," "removing this edge/node disconnects the graph."
 - Core idea: one DFS tracking `discovery[]` and `low[]` (lowest discovery time reachable, including via one back-edge) per node. An edge `(u, v)` is a **bridge** if `low[v] > disc[u]`. A node `u` is an **articulation point** if it's the DFS root with 2+ children, or any non-root child `v` has `low[v] >= disc[u]`.
-

Java Templates for Major Algorithms

BFS (unweighted shortest path)

```
public int bfs(int start, int target, Map<Integer, List<Integer>> adj) {
    Queue<Integer> queue = new LinkedList<>();
    Set<Integer> visited = new HashSet<>();
    queue.offer(start); visited.add(start);
    int dist = 0;
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            int node = queue.poll();
            if (node == target) return dist;
            for (int nxt : adj.getOrDefault(node, List.of())) {
                if (visited.add(nxt)) queue.offer(nxt);
            }
        }
        dist++;
    }
    return -1;
}
```

DFS (recursive)

```
void dfs(int node, Map<Integer, List<Integer>> adj, Set<Integer> visited) {
    visited.add(node);
    for (int neighbor : adj.getOrDefault(node, List.of())) {
        if (!visited.contains(neighbor)) dfs(neighbor, adj, visited);
    }
}
```

Dijkstra's Algorithm

```

public int[] dijkstra(int n, List<int[]>[] adj, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // {node, d
ist}
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int node = cur[0], d = cur[1];
        if (d > dist[node]) continue; // stale entry
        for (int[] edge : adj[node]) {
            int next = edge[0], weight = edge[1];
            if (dist[node] + weight < dist[next]) {
                dist[next] = dist[node] + weight;
                pq.offer(new int[]{next, dist[next]});
            }
        }
    }
    return dist;
}

```

Bellman-Ford

```

public int[] bellmanFord(int n, int[][] edges, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) {
        for (int[] e : edges) {
            int u = e[0], v = e[1], w = e[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    // Optional: one more pass - if anything still relaxes, a negative cycle exists
    for (int[] e : edges) {
        int u = e[0], v = e[1], w = e[2];
        if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
            throw new RuntimeException("Negative cycle detected");
        }
    }
    return dist;
}

```

Floyd-Warshall

```

public void floydWarshall(int[][] dist, int n) {
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != Integer.MAX_VALUE && dist[k][j] != Integer.MAX_VALU
E
                    && dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
            }
        }
    }
}

```

0-1 BFS

```

public int zeroOneBFS(int n, List<int[]>[] adj, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    Deque<Integer> deque = new ArrayDeque<>();
    deque.offerFirst(src);
    while (!deque.isEmpty()) {
        int node = deque.pollFirst();
        for (int[] edge : adj[node]) {
            int next = edge[0], weight = edge[1];    // weight is 0 or 1
            if (dist[node] + weight < dist[next]) {
                dist[next] = dist[node] + weight;
                if (weight == 0) deque.offerFirst(next);
                else deque.offerLast(next);
            }
        }
    }
    return dist[n - 1];
}

```

Union-Find (DSU) with path compression + union by rank

```

class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);    // path compression
        return parent[x];
    }
    boolean union(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;    // already connected - this edge would create a
cycle
        if (rank[rx] < rank[ry]) { int t = rx; rx = ry; ry = t; }
        parent[ry] = rx;
        if (rank[rx] == rank[ry]) rank[rx]++;
        return true;
    }
}

```

Kruskal's Algorithm (MST)

```

public int kruskal(int n, int[][] edges) {
    Arrays.sort(edges, (a, b) -> a[2] - b[2]);    // sort by weight
    DSU dsu = new DSU(n);
    int cost = 0, edgesUsed = 0;
    for (int[] e : edges) {
        if (dsu.union(e[0], e[1])) {
            cost += e[2];
            edgesUsed++;
        }
    }
    return edgesUsed == n - 1 ? cost : -1;    // -1 if graph can't be fully connected
}

```

Prim's Algorithm (MST)

```

public int prim(int n, List<int[]>[] adj) {
    boolean[] inMST = new boolean[n];
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // {node, weight}
    pq.offer(new int[]{0, 0});
    int cost = 0, nodesAdded = 0;
    while (!pq.isEmpty() && nodesAdded < n) {
        int[] cur = pq.poll();
        int node = cur[0], weight = cur[1];
        if (inMST[node]) continue;
        inMST[node] = true;
        cost += weight;
        nodesAdded++;
        for (int[] edge : adj[node]) {
            if (!inMST[edge[0]]) pq.offer(edge);
        }
    }
    return nodesAdded == n ? cost : -1;
}

```

Kahn's Algorithm (Topological Sort)

```

public int[] topoSort(int n, int[][] edges) {
    List<List<Integer>> adj = new ArrayList<>();
    int[] indegree = new int[n];
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (int[] e : edges) { adj.get(e[0]).add(e[1]); indegree[e[1]]++; }
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) if (indegree[i] == 0) queue.offer(i);
    int[] order = new int[n];
    int idx = 0;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        order[idx++] = node;
        for (int nxt : adj.get(node)) if (--indegree[nxt] == 0) queue.offer(nxt);
    }
    return idx == n ? order : new int[0]; // empty = cycle exists
}

```

Bridges (Tarjan's Algorithm)

```

int timer = 0;

void findBridges(int node, int parent, List<Integer>[] adj, int[] disc, int[] low,
                boolean[] visited, List<int[]> bridges) {
    visited[node] = true;
    disc[node] = low[node] = timer++;
    for (int next : adj[node]) {
        if (next == parent) continue;
        if (visited[next]) {
            low[node] = Math.min(low[node], disc[next]);
        } else {
            findBridges(next, node, adj, disc, low, visited, bridges);
            low[node] = Math.min(low[node], low[next]);
            if (low[next] > disc[node]) {
                bridges.add(new int[]{node, next}); // (node, next) is a bridge
            }
        }
    }
}

```

Top 30 Must-Do Graph Problems

1. LC 200 — Number of Islands
2. LC 133 — Clone Graph
3. LC 994 — Rotting Oranges
4. LC 542 — 01 Matrix
5. LC 1091 — Shortest Path in Binary Matrix
6. LC 547 — Number of Provinces
7. LC 684 — Redundant Connection
8. GfG — Detect cycle in an undirected graph
9. LC 207 — Course Schedule
10. GfG — Detect Cycle in a Directed Graph
11. LC 210 — Course Schedule II
12. LC 269 — Alien Dictionary 🔒
13. LC 785 — Is Graph Bipartite?
14. LC 743 — Network Delay Time
15. LC 1631 — Path With Minimum Effort
16. LC 787 — Cheapest Flights Within K Stops
17. GfG — Negative weight cycle

18. LC 1334 — Find the City With the Smallest Number of Neighbors at a Threshold Distance
19. LC 1368 — Minimum Cost to Make at Least One Valid Path in a Grid
20. LC 990 — Satisfiability of Equality Equations
21. LC 1584 — Min Cost to Connect All Points
22. GfG — Minimum Spanning Tree
23. LC 1192 — Critical Connections in a Network
24. GfG — Articulation Points (or Cut Vertices) in a Graph
25. GfG — Strongly Connected Components (Kosaraju's Algorithm)
26. GfG — M-Coloring Problem
27. GfG — Hamiltonian Path
28. LC 847 — Shortest Path Visiting All Nodes
29. LC 721 — Accounts Merge
30. LC 2115 — Find All Possible Recipes from Given Supplies

Top 50 Interview Problems

(The 30 above, plus:)

1. LC 695 — Max Area of Island
2. LC 841 — Keys and Rooms
3. LC 1466 — Reorder Routes to Make All Paths Lead to the City Zero
4. LC 1926 — Nearest Exit from Entrance in Maze
5. LC 934 — Shortest Bridge
6. LC 1319 — Number of Operations to Make Network Connected
7. LC 802 — Find Eventual Safe States
8. LC 310 — Minimum Height Trees
9. LC 886 — Possible Bipartition
10. LC 1976 — Number of Ways to Arrive at Destination
11. LC 399 — Evaluate Division
12. LC 2290 — Minimum Obstacle Removal to Reach Corner
13. LC 947 — Most Stones Removed with Same Row or Column
14. LC 1489 — Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree
15. LC 797 — All Paths From Source to Target
16. LC 2492 — Minimum Score of a Path Between Two Cities
17. LC 2246 — Longest Path With Different Adjacent Characters
18. LC 815 — Bus Routes
19. LC 1595 — Minimum Cost to Connect Two Groups of Points
20. LC 943 — Find the Shortest Superstring

Beginner → Advanced Complete Sequence

Follow the Master Roadmap tables above, Level 0 → Level 10, Order 1 → 88. Within a level, problems are already sequenced so each builds on the one before it.

Final Summary

Total curated problems: 88 unique (80 LeetCode + 8 GeeksforGeeks — zero duplication across platforms)

Platform	Count
LeetCode	80
GeeksforGeeks	8

Difficulty	Count (approx.)
Easy	6
Medium	55
Hard	27

Level / Pattern Group	# of Problems
Level 0 — Fundamentals & Representation	3
Level 1 — BFS/DFS Basics	7
Level 2 — Grid & Matrix Graphs	12
Level 3 — Connected Components & Cycle Detection	10
Level 4 — Topological Sort / Kahn's	8 new (+2 cross-ref)
Level 5 — Bipartite Graphs	4
Level 6 — Shortest Path (all 5 sub-patterns)	14 new (+3 cross-ref)
Level 7 — Union Find / MST	12 new (+1 cross-ref)
Level 8 — Advanced Graph Algorithms	9
Level 9 — Graph + DP/Greedy/Heap	6 new (+2 cross-ref)
Level 10 — Advanced/Hard Capstones	3 new (+5 cross-ref)

Tag	Count (approx.)
🔥 Must-Do	39
★ Recommended	24
💡 Optional	4
🏆 Hard Challenge	21

*Note on Level 5 (Bipartite Graphs) and Level 8's Hamiltonian/Coloring sub-patterns: these sections are intentionally smaller than others (4 and 2–3 respectively). LeetCode and GfG combined simply don't have 8+ *distinct, non-redundant quality problems for these narrower patterns — padding them with repetitive or borderline-irrelevant problems would violate the "don't invent/pad" requirement, so depth was prioritized over a forced quota here.**

Final Revision List — Most Important Problems to Re-Solve Before an Interview

One per core algorithm/pattern, chosen so each forces recall of a genuinely distinct template:

1. **LC 200** — Number of Islands (grid BFS/DFS mechanics)
2. **LC 994** — Rotting Oranges (multi-source BFS — seed the queue with every source)
3. **LC 547** — Number of Provinces (connected components)
4. **LC 684** — Redundant Connection (DSU-based cycle detection)
5. **GfG — Detect Cycle in a Directed Graph** (DFS-coloring cycle detection)
6. **LC 207 / 210** — Course Schedule I & II (Kahn's Algorithm)
7. **LC 785** — Is Graph Bipartite? (2-coloring BFS/DFS)
8. **LC 743** — Network Delay Time (Dijkstra — heap-based relaxation)
9. **LC 787** — Cheapest Flights Within K Stops (Bellman-Ford — bounded relaxations)
10. **LC 1334** — Find the City... Threshold Distance (Floyd-Warshall — all-pairs DP)
11. **LC 1368** — Minimum Cost to Make at Least One Valid Path in a Grid (0-1 BFS)
12. **LC 990** — Satisfiability of Equality Equations (DSU fundamentals)
13. **LC 1584** — Min Cost to Connect All Points (Kruskal's/Prim's — MST)
14. **LC 1192** — Critical Connections in a Network (Tarjan's — bridges)
15. **GfG — Strongly Connected Components (Kosaraju's Algorithm)** (two-pass DFS on transpose)
16. **LC 847** — Shortest Path Visiting All Nodes (BFS + bitmask state)

Accuracy note: LeetCode numbers/titles were verified against LeetCode's own problem pages and cross-checked against independent, current source listings, including explicit confirmation of premium status for LC 269, 286-family, 323, 261, 1135, and 1168. GfG problem titles were verified against GfG's

practice-page and topic-tag listings at the time of writing. No problem in this roadmap appears on more than one platform's entry — where a pattern exists on both LeetCode and GfG, GfG was included only when it taught a distinct algorithmic approach (e.g., DFS-based vs. DSU-based cycle detection) rather than a redundant restatement of the same LeetCode problem.